



Faculty of Engineering and Technology
Electrical and Computer Engineering Department
DATA BASE SYSTEMS (COMP333)
Final Project Report

Student 1 name: Samar Hmedan

Student 1 number:1230925

Student 2 name: Rahaf Salahat

Student 2 number:1231584

Instructor: Bassem Sayrafi

Section: 1

Date: 21th June 2026

Contents

Table of figures	2
1. Project Description	3
2. Main Functionality.....	3
3.Client Information	4
4. Entities Description.....	5
5. Relationships Description:	11
6. ER Diagram	14
7. Relational Schema:	16
8. Queries:.....	18
9. Normalization:.....	20
10. Conclusion:	21

Table of figures

Figure 1:Entity Relationship Diagram (ERD) of the Alawneh Car Trading Company Database System.....	14
--	----

1. Project Description

Alawneh Car Trading Company Database System is a comprehensive database application designed to support the daily operations of a car trading company that specializes in selling vehicles and spare parts through multiple branches. The system provides a centralized platform for managing branches, warehouses, vehicles, spare parts, employees, suppliers, customers, sales transactions, purchase transactions, and payment records.

The company operates through multiple branches where cars are displayed for customers, while spare parts are stored in dedicated warehouses associated with each branch. To ensure efficient business operations, the system supports both sales and purchasing activities, allowing the company to maintain sufficient stock levels and accurately track all transactions.

A major feature of the system is inventory management, which enables the company to monitor spare part quantities in different warehouses. In addition, the system records stock movements resulting from purchases, sales, and transfers, providing complete traceability of inventory operations and improving inventory control.

The system also supports transfer operations between company locations. Cars can be transferred between branches, while spare parts can be transferred between warehouses when needed. Furthermore, each vehicle is uniquely identified using a Vehicle Identification Number (VIN), which ensures accurate vehicle tracking and management throughout the system.

Overall, the proposed database system aims to improve operational efficiency, enhance data accuracy, reduce manual work, and provide management with reliable information for decision-making and business monitoring.

2. Main Functionality

The scope of this project includes designing and implementing a complete database management system for Alawneh Car Trading Company. The system is intended to support the company's core business operations by managing vehicles, spare parts, branches, warehouses, employees, suppliers, customers, and financial transactions.

The system provides functionalities for managing car inventories available in different branches and spare parts stored in warehouses. It supports sales and purchase operations, records payment transactions, and maintains detailed information about customers and suppliers. In addition, the system enables employees and administrators to efficiently monitor and update operational data.

The system also records which employee added each car and spare part, allowing better auditing and tracking of operational activities.

To improve inventory control, the system includes inventory management and stock movement tracking. These features allow the company to monitor spare part quantities, track stock changes resulting from purchases and sales, and maintain a complete history of inventory activities.

The project focuses on supporting internal business processes and does not include online shopping services, customer account registration, or e-commerce functionalities.

3.Client Information

Client Name: Alawneh Car Trading Company

Business Type: Automotive Trading Company

Location: Palestine (Ramallah and Jenin Branches)

Description:

Alawneh Car Trading Company is a medium-sized automotive business that specializes in selling vehicles and spare parts. The company operates through multiple branches where cars are displayed for customers, while spare parts are stored and managed through warehouses associated with each branch.

The company's daily operations include managing vehicle inventories, tracking spare parts, processing sales transactions, purchasing products from suppliers, monitoring payments, and controlling inventory movement between locations. Each branch is connected to a warehouse that supports inventory management and spare part storage activities.

The system is intended to be used by employees and administrators. Employees perform daily operational tasks such as managing cars and spare parts,

recording sales and purchases, handling inventory transactions, and processing transfers between branches and warehouses. Administrators have broader access to manage users and roles, monitor records, review transactions, and supervise overall system performance.

The primary objective of the client is to replace manual and scattered record-keeping methods with a centralized database system that improves data accuracy, operational efficiency, inventory control, and decision-making.

4. Entities Description

1. Branch

Purpose: Stores information about company branches where vehicles are displayed and sold.

Attributes:

- branch_id
- branch_name
- city
- physical_location

2. Warehouse

Purpose: Stores information about warehouses used to store spare parts.

Attributes:

- warehouse_id
- location
- capacity
- branch_id (FK)

3. Brand

Purpose: Stores information about vehicle and spare part brands.

Attributes:

- brand_id

- **brand_name**

4. CarCategory

Purpose: Stores different categories of vehicles.

Attributes:

- car_cat_id
- car_cat_name

5. PartCategory

Purpose: Stores different categories of spare parts.

Attributes:

- part_cat_id
- part_cat_name

6. Car

Purpose: Stores information about vehicles available in the company.

Attributes:

- | | |
|------------------------|-------------------|
| • car_id | • price |
| • vin | • image_path |
| • model | • branch_id (FK) |
| • manufacture_year | • brand_id (FK) |
| • color | • car_cat_id (FK) |
| • added_by_emp_id (FK) | |
| • status | |

7. SparePart

Purpose: Stores information about spare parts sold and managed by the company.

Attributes:

- | | |
|-------------|---------------|
| • part_id | • part_number |
| • part_name | • status |

- added_by_emp_id (FK)
- image_path
- unit_price
- brand_id (FK)
- part_cat_id (FK)

8. Inventory

Purpose: Stores information about spare part quantities available in warehouses.

Attributes:

- inventory_id
- quantity
- date_in
- date_out
- warehouse_id (FK)
- part_id (FK)

9. Customer

Purpose: Stores information about customers who purchase cars or spare parts.

Attributes:

- cust_id
- cust_name
- address
- email

10. Supplier

Purpose: Stores information about suppliers who provide vehicles and spare parts.

Attributes:

- sup_id
- sup_name
- address

11. Employee

Purpose: Stores information about company employees who manage system operations.

Attributes:

- emp_id
- emp_name
- salary
- job_title
- role_id (FK)
- user_id (FK)
- branch_id (FK)

12. Users

Purpose: Stores login account information used to access the system.

Attributes:

- user_id
- email
- password
- is_active

13. Role

Purpose: Stores information about system roles and permissions.

Attributes:

- role_id
- role_name

14. Sales

Purpose: Stores information about sales transactions performed by customers.

Attributes:

- sale_id
- sale_date
- sale_price
- branch_id (FK)
- cust_id (FK)
- emp_id (FK)

15. SaleCarDetails

Purpose: Stores detailed information about cars included in sales transactions.

Attributes:

- sale_car_details_id
- quantity
- unit_price
- sale_id (FK)
- car_id (FK)

16. SalePartDetails

Purpose:

Stores detailed information about spare parts included in sales transactions.

Attributes:

- sale_part_details_id
- quantity
- unit_price
- warehouse_id (FK)
- sale_id (FK)
- part_id (FK)

17. SalePayment

Purpose: Stores payment information related to sales transactions.

Attributes:

- payment_id
- payment_date
- amount
- payment_type
- sale_id (FK)

18. Purchase

Purpose: Stores information about purchase transactions made from suppliers.

Attributes:

- purchase_id
- purchase_date
- cost
- emp_id (FK)
- sup_id (FK)

19. PurchaseCarDetails

Purpose: Stores information about vehicles acquired through purchase transactions.

Attributes:

- purchase_car_details_id
- quantity
- unit_price
- purchase_id (FK)
- car_id (FK)

20. PurchasePartDetails

Purpose: Stores information about spare parts acquired through purchase transactions.

Attributes:

- purchase_part_details_id
- quantity
- unit_price
- purchase_id (FK)
- part_id (FK)

21. PurchasePayment

Purpose: Stores payment information related to purchase transactions.

Attributes:

- payment_id
- payment_date
- amount
- payment_type
- purchase_id (FK)

22. StockMovement

Purpose: Stores the history of inventory changes.

Attributes:

- movement_id
- movement_type
- quantity
- movement_date
- notes
- warehouse_id (FK)
- part_id (FK)

23. CarTransferDetails

Purpose: Stores information about vehicle transfers between branches.

Attributes:

- car_transfer_id
- transfer_date
- status
- notes
- emp_id (FK)
- car_id (FK)

- from_branch_id (FK)
- to_branch_id (FK)

24. SparePartTransferDetails

Purpose: Stores information about spare part transfers between warehouses.

Attributes:

- spare_transfer_id
- quantity
- transfer_date
- status
- emp_id (FK)
- notes
- part_id (FK)
- from_warehouse_id (FK)
- to_warehouse_id (FK)

5. Relationships Description:

Branch – Warehouse (1:1): Each branch has exactly one warehouse, and each warehouse belongs to one branch.

Branch – Car (1:M): Each branch may contain many cars, while each car belongs to one branch.

Branch – Employee (1:M): Each branch may have many employees, while each employee works in one branch.

CarCategory – Car (1:M): Each car category may include many cars, while each car belongs to one category.

Brand – Car (1:M): Each brand may be associated with many cars, while each car belongs to one brand.

Brand – SparePart (1:M): Each brand may be associated with many spare parts, while each spare part belongs to one brand.

PartCategory – SparePart (1:M): Each part category may include many spare parts, while each spare part belongs to one category.

Warehouse – Inventory (1:M): Each warehouse may have many inventory records, while each inventory record belongs to one warehouse.

SparePart – Inventory (1:M): Each spare part may appear in many inventory records, while each inventory record refers to one spare part.

Warehouse – SparePart through Inventory (M:N): A warehouse can store many spare parts, and the same spare part can be stored in many warehouses. This relationship is implemented through the Inventory table.

Warehouse – StockMovement (1:M): Each warehouse may have many stock movement records, while each stock movement belongs to one warehouse.

Warehouse – SalePartDetails (1:M): Each warehouse may appear in many sale part detail records, while each sale part detail record refers to one warehouse.

SparePart – StockMovement (1:M): Each spare part may have many stock movement records, while each stock movement refers to one spare part.

Customer – Sales (1:M): Each customer may make many sales transactions, while each sale belongs to one customer.

Employee – Car (1:M): Each employee may add many cars, while each car is added by one employee.

Employee – SparePart (1:M): Each employee may add many spare parts, while each spare part is added by one employee.

Employee – Sales (1:M): Each employee may handle many sales transactions, while each sale is handled by one employee.

Sales – SaleCarDetails (1:M): Each sale may include many car detail records, while each SaleCarDetails record belongs to one sale.

Car – SaleCarDetails (1:M): Each car may appear in sale car details, while each SaleCarDetails record refers to one car.

Sales – SalePartDetails (1:M): Each sale may include many spare part detail records, while each SalePartDetails record belongs to one sale.

Branch – Sales (1:M): Each branch may have many sales, while each sale belongs to one branch.

SparePart – SalePartDetails (1:M): Each spare part may appear in many sale part detail records, while each SalePartDetails record refers to one spare part.

Sales – SalePayment (1:M): Each sale may have one or more payment records, while each payment belongs to one sale.

Supplier – Purchase (1:M): Each supplier may provide many purchase transactions, while each purchase belongs to one supplier.

Employee – Purchase (1:M): Each employee may handle many purchase transactions, while each purchase is handled by one employee.

Employee – CarTransferDetails (1:M): Each employee may handle many car transfer records, while each car transfer record is handled by one employee.

Purchase – PurchaseCarDetails (1:M): Each purchase may include many purchased car detail records, while each PurchaseCarDetails record belongs to one purchase.

Car – PurchaseCarDetails (1:M): Each car may appear in purchase car details, while each PurchaseCarDetails record refers to one car.

Purchase – PurchasePartDetails (1:M): Each purchase may include many purchased spare part detail records, while each PurchasePartDetails record belongs to one purchase.

SparePart – PurchasePartDetails (1:M): Each spare part may appear in many purchase part detail records, while each PurchasePartDetails record refers to one spare part.

Purchase – PurchasePayment (1:M): Each purchase may have one or more payment records, while each payment belongs to one purchase.

Role – Employee (1:M): Each role may be assigned to many employees, while each employee has one role.

Users – Employee (1:1): Each user account is associated with one employee, and each employee has one user account.

Car – CarTransferDetails (1:M): Each car may have many transfer records, while each transfer record refers to one car.

Branch – CarTransferDetails as Sending Branch (1:M): Each branch may send many cars, while each transfer has one sending branch.

Branch – CarTransferDetails as Receiving Branch (1:M): Each branch may receive many cars, while each transfer has one receiving branch.

SparePart – SparePartTransferDetails (1:M): Each spare part may have many transfer records, while each transfer record refers to one spare part.

employee responsible for adding cars and spare parts through the attribute `added_by_emp_id`.

Relationships between entities were established according to the business rules of the system. One-to-one relationships were implemented by placing a foreign key in one of the participating entities. One-to-many relationships were represented by storing the primary key of the parent entity as a foreign key in the child entity. Many-to-many relationships were resolved using associative entities such as `PurchaseCarDetails`, `PurchasePartDetails`, `SaleCarDetails`, `SalePartDetails`, and `Inventory`, which contain the foreign keys of both participating entities.

No multivalued attributes were stored directly inside entities. Whenever an attribute could contain multiple values, a separate entity was created to preserve normalization and avoid redundancy. For example, supplier phone numbers were separated into the `SupplierPhone` entity instead of storing multiple phone numbers inside the `Supplier` entity.

Different cardinalities were used during the ER design. One-to-one relationships were used when each entity instance corresponded to exactly one instance in another entity. One-to-many relationships were used when one record could be associated with several records in another entity, such as `Branch–Car` and `Employee–Car`. Many-to-many relationships were transformed into intermediate entities to maintain database normalization and support additional attributes associated with these relationships.

7. Relational Schema:

Branch(
branch_id PK,
branch_name,
city,
physical_location
)

Warehouse(
warehouse_id PK,
location,
capacity,
branch_id FK
)

Brand(
brand_id PK,
brand_name
)

CarCategory(
car_cat_id PK,
car_cat_name
)

PartCategory(
part_cat_id PK,
part_cat_name
)

Car(
car_id PK,
vin,
model,
manufacture_year,
color,
status,
price,
branch_id FK,
brand_id FK,

car_cat_id FK,
added_by_emp_id FK,
image_path
)

SparePart(
part_id PK,
part_name,
part_number,
status,
unit_price,
brand_id FK,
part_cat_id FK,
image_path,
added_by_emp_id FK
)

Inventory(
inventory_id PK,
quantity,
date_in,
date_out,
warehouse_id FK,
part_id FK
)

Role(
role_id PK,
role_name
)

Users(
user_id PK,
email,
password,
is_active
)

Employee(
emp_id PK,
emp_name,
salary,

job_title,
role_id FK,
user_id FK,
branch_id FK
)

Customer(
cust_id PK,
cust_name,
address,
email
)

Supplier(
sup_id PK,
sup_name,
address
)

Sales(
sale_id PK,
saleDate,
salePrice,
cust_id FK,
emp_id FK,
branch_id FK
)

SaleCarDetails(
sale_car_details_id PK,
quantity,
unit_price,
sale_id FK,
car_id FK
)

SalePartDetails(
sale_part_details_id PK,
quantity,
unit_price,
sale_id FK,
part_id FK,
warehouse_id FK
)

SalePayment(
payment_id PK,
paymentDate,
amount,
paymentType,
sale_id FK
)

Purchase(
purchase_id PK,
purchaseDate,
cost,
emp_id FK,
sup_id FK
)

PurchaseCarDetails(
purchase_car_details_id PK,
quantity,
unit_price,
purchase_id FK,
car_id FK
)

PurchasePartDetails(
purchase_part_details_id PK,
quantity,
unit_price,
purchase_id FK,
part_id FK
)

PurchasePayment(
payment_id PK,
paymentDate,
amount,
payment_type,
purchase_id FK
)

StockMovement(
movement_id PK,
movement_type,
quantity,
movement_date,

```
notes,  
warehouse_id FK,  
part_id FK  
)
```

```
CarTransferDetails(  
car_transfer_id PK,  
transfer_date,  
status,  
notes,  
car_id FK,  
from_branch_id FK,  
to_branch_id FK,  
  
emp_id FK  
)
```

```
SparePartTransferDetails(  
spare_transfer_id PK,  
quantity,  
transfer_date,  
status,  
notes,  
part_id FK,  
from_warehouse_id FK,  
to_warehouse_id FK,  
  
emp_id FK  
)
```

8. Queries:

The following queries demonstrate the main functionalities supported by the proposed database system.

1. Retrieve all branches located in a specific city.
2. Retrieve the warehouse information for each branch.
3. Retrieve all cars available in a specific branch.
4. Retrieve all cars belonging to a specific brand in a specific branch.
5. Retrieve all cars that belong to a specific car category.
6. Retrieve all available cars within a specific price range.
7. Retrieve all cars manufactured within a specific range of years.
8. Retrieve all cars currently marked as available.
9. Retrieve all cars currently marked as sold.
10. Retrieve the most recently added cars in the system.
11. Retrieve the total number of cars available in each branch.
12. Retrieve the average car price for each branch.
13. Retrieve all spare parts stored in a specific warehouse.
14. Retrieve all spare parts belonging to a specific brand.

15. Retrieve all spare parts belonging to a specific part category.
16. Retrieve the quantity of each spare part in each warehouse.
17. Retrieve all spare parts available in the warehouse of a specific branch.
18. Retrieve spare parts whose quantity is below a specific threshold.
19. Retrieve the total number of spare parts stored in each warehouse.
20. Retrieve all stock movements for a specific spare part.
21. Retrieve all stock movements within a specific date range.
22. Retrieve the total quantity of spare parts added through purchases.
23. Retrieve the total quantity of spare parts removed through sales.
24. Retrieve all stock movements performed in a specific warehouse.
25. Retrieve all customers who purchased cars during a specific period.
26. Retrieve all customers who purchased spare parts during a specific period.
27. Retrieve all sales transactions handled by a specific employee.
28. Retrieve all sale payments related to a specific sale transaction.
29. Retrieve the total sales amount in the system.
30. Retrieve all purchase transactions handled by a specific employee.
31. Retrieve all purchase transactions made from a specific supplier.
32. Retrieve all purchase payments related to a specific purchase transaction.
33. Retrieve the total purchase cost from each supplier.
34. Retrieve all car transfers between branches.
35. Retrieve all pending car transfers.
36. Retrieve all spare part transfers between warehouses.
37. Retrieve all completed spare part transfers.
38. Retrieve all employees and their assigned roles.
39. Retrieve all active users in the system.
40. Retrieve employees working in a specific branch.
41. Retrieve all system users together with their employee information and roles.
42. Retrieve all cars together with the employee who added them.
43. Retrieve all spare parts together with the employee who added them.

44. Retrieve car transfers handled by a specific employee.
45. Retrieve spare part transfers handled by a specific employee.
46. Retrieve the number of cars added by each employee.
47. Retrieve the number of spare parts added by each employee.

9. Normalization:

9.1 First Normal Form (1NF):

All relations satisfy the First Normal Form (1NF) because each attribute contains atomic values only, and there are no repeating groups or multi-valued attributes. Each table stores a single value in every field, and all data items are represented in separate attributes.

9.2 Second Normal Form (2NF):

The database schema satisfies Second Normal Form (2NF). Since all tables use single-attribute primary keys, partial functional dependencies are not possible. Therefore, all non-key attributes depend fully on the primary key, and no violations of 2NF exist.

9.3 Third Normal Form (3NF):

The database schema is normalized to Third Normal Form (3NF). Since all relations already satisfy 2NF, they were further analyzed for transitive dependencies. No non-key attribute depends on another non-key attribute. Entity-specific information is stored in separate tables and linked through foreign keys. For example, brand information is stored in the Brand table, role information is stored in the Role table, and category information is stored in CarCategory and PartCategory tables. The addition of `added_by_emp_id` in Car and SparePart, `branch_id` in Sales, `warehouse_id` in SalePartDetails, and `emp_id` in transfer details does not violate 3NF because these attributes are foreign keys that reference related entities and do not introduce partial or transitive dependencies. This design eliminates transitive dependencies and reduces data redundancy.

10. Conclusion:

In conclusion, the Alawneh Car Trading Company Database System provides a structured and centralized solution for managing the company's main operations. The system supports the management of branches, warehouses, cars, spare parts, employees, customers, suppliers, sales, purchases, payments, inventory, stock movements, and transfer operations.

The proposed database design helps reduce manual work, improve data accuracy, and organize company records in a clear and reliable way. By using primary keys and foreign keys, the system maintains strong relationships between entities and ensures data consistency across different operations.

The system also supports important business functions such as tracking car availability, monitoring spare part quantities, recording sales and purchase transactions, and managing transfers between branches and warehouses. In addition, the database schema was normalized up to Third Normal Form (3NF), which reduces redundancy and helps avoid insertion, deletion, and update anomalies. In addition, the system provides traceability features by recording the employee responsible for adding cars and spare parts, which improves accountability and supports better monitoring of business activities.

Overall, this project demonstrates how a well-designed relational database can improve operational efficiency, support decision-making, and provide better control over the daily activities of Alawneh Car Trading Company.